CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 14: Evaluate Accuracy-Latency Trade-Offs

Exam objective: Evaluate trade-offs between accuracy and latency in Claude-based systems, including model selection, reasoning effort, retrieval depth, reranking, validation and retries, parallel tool execution, streaming, prompt caching, and performance benchmarking.

Q1 **A real-time trading-floor assistant must respond to simple position-lookup queries in under 500ms, but also occasionally handles complex multi-step portfolio risk analysis. What is the best architecture to balance accuracy and latency?**

A Use the largest, highest-reasoning-effort model for every query to guarantee accuracy

B **Route simple lookups to a smaller/faster model or low reasoning-effort configuration, and dynamically escalate to a larger model or higher reasoning effort only for complex analytical queries**

C Always use the smallest model to guarantee the lowest latency, accepting lower accuracy

D Cache every possible query result in advance so no model call is ever needed

CORRECT ANSWER **B**

WHY Dynamic routing/escalation based on query complexity is the standard pattern for balancing latency and accuracy — simple requests get fast, "good enough" answers while complex ones get the reasoning depth they need. Always using the largest model wastes latency budget on trivial queries, always using the smallest model sacrifices accuracy on complex ones, and pre-caching is infeasible for open-ended financial queries.

Q2 **A customer support agent for an e-commerce platform retrieves 50 document chunks per query to maximize the chance of finding relevant context, but this adds 2+ seconds of latency and sometimes confuses the model with irrelevant chunks. What change best improves both accuracy and latency?**

A Increase retrieval depth further to 100 chunks to be even more thorough

B Reduce retrieval depth to a smaller top-k, paired with a reranking step that selects the most relevant chunks before they reach the model

C Remove retrieval entirely and rely only on the model's parametric knowledge

D Keep 50 chunks but ask the model to ignore irrelevant ones via prompt instructions

CORRECT ANSWER B

WHY Reducing initial retrieval breadth and adding a reranking stage to surface only the most relevant chunks typically improves both latency (fewer tokens to process) and accuracy (less irrelevant context diluting the model's attention), which is a well-established RAG optimization. Increasing chunk count worsens both metrics, removing retrieval loses access to proprietary/current data, and relying on prompt instructions to "ignore" noise is unreliable and still costs the tokens/latency of processing it.

Q3 **A manufacturing quality-control agent needs to call five independent diagnostic tools (temperature sensor, vibration sensor, pressure sensor, vision inspection, and historical log lookup) to produce one report. Currently these are called sequentially, adding significant latency. What is the best optimization?**

A Combine all five into a single tool with no parameters to reduce call count

B **Execute the independent tool calls in parallel where the model/orchestration layer supports concurrent tool execution, then synthesize results once all complete**

C Remove two of the five tools to reduce latency, accepting lower report completeness

D Cache the results of all five tools indefinitely regardless of sensor readings changing

CORRECT ANSWER **B**

WHY When tool calls are independent of each other's outputs, executing them in parallel reduces end-to-end latency without sacrificing the completeness that comes from calling all five, which is the core value of parallel tool execution support. Merging into one opaque tool harms modularity and reuse, dropping tools reduces accuracy/completeness, and indefinitely caching live sensor data produces stale, inaccurate readings.

Q4 **A healthcare documentation assistant streams Claude's response token-by-token to the clinician's UI. A product manager asks whether this improves actual end-to-end latency of the full response. What is the accurate answer?**

A Streaming reduces the total time to generate the full response

B **Streaming does not reduce total generation time, but it significantly improves perceived latency/responsiveness by showing output incrementally as it's produced**

C Streaming only works for the first 100 tokens of any response

D Streaming increases total generation time because of added network overhead

CORRECT ANSWER **B**

WHY Streaming is primarily a perceived-latency (UX) optimization — the user sees partial output immediately rather than waiting for the full response — but it does not change the underlying token generation throughput or total compute time. It is not limited to a fixed token count, and while there is some overhead, the main trade-off discussed in architecture contexts is the UX benefit, not a meaningful increase in total generation time.

Q5 An enterprise deploys a Claude agent where the same large system prompt (tool definitions, policies, few-shot examples) is sent on every single request, causing redundant processing and cost. Which technique directly addresses this without sacrificing accuracy?

A Shorten the system prompt by removing tool definitions entirely

B Use prompt caching so the static portions of the prompt (system instructions, tool schemas) are cached and reused across requests, avoiding reprocessing of unchanged content

C Switch to a model with no system prompt support

D Send the system prompt only on every other request to save bandwidth

CORRECT ANSWER B

WHY Prompt caching is designed exactly for this scenario: stable, repeated prefix content (system prompts, tool definitions) is cached so subsequent requests skip reprocessing that portion, cutting latency and cost while preserving full instruction fidelity. Removing tool definitions breaks functionality, switching models to avoid system prompts isn't a real latency fix, and omitting the system prompt on alternating requests would cause inconsistent/incorrect behavior.

Q6 **A logistics company's route-planning agent occasionally returns malformed JSON from a tool-calling step, causing downstream parsing failures. The team wants to improve reliability without materially harming latency for the common case. What is the best approach?**

A Always run the request three times in parallel and pick a result at random

B **Add lightweight output validation with a single targeted retry (re-prompt or regenerate) only when validation fails, rather than retrying every request by default**

C Increase reasoning effort to maximum for every request to eliminate all malformed output

D Remove JSON formatting requirements entirely and parse free text with regex

CORRECT ANSWER **B**

WHY Validate-then-retry-on-failure is the standard low-overhead reliability pattern: most requests succeed on the first try and incur no extra latency, while the rare failures get a targeted correction. Always running three parallel requests wastes cost/latency on every call, maximizing reasoning effort for all requests is latency-expensive and doesn't guarantee valid JSON, and abandoning structured output for regex parsing trades one reliability problem for a worse one.

Q7 **A SaaS analytics company is benchmarking two configurations: Config A (smaller model, higher retrieval depth) and Config B (larger model, shallower retrieval) for a customer-facing Q&A feature. What is the most rigorous way to decide between them?**

A Pick the configuration the engineering team personally prefers

B **Run both configurations against a representative, labeled evaluation set measuring accuracy (e.g., correctness, groundedness) and latency/cost metrics, then choose based on the organization's defined trade-off priorities**

C Always choose the larger model configuration since bigger models are always better

D Choose based solely on which configuration has the lowest average token count per response

CORRECT ANSWER **B**

WHY Rigorous performance benchmarking requires representative evaluation data and multi-dimensional measurement (accuracy and latency/cost) weighed against explicit business priorities, since there is rarely a configuration that wins on every axis. Personal preference is not a rigorous method, bigger models aren't categorically better for every task/latency budget, and token count alone ignores accuracy entirely.

Q8 **A government services chatbot needs to decide whether to use Claude's extended/high reasoning effort mode for every citizen query, including simple FAQ-style questions like "What are your office hours?" What is the most appropriate policy?**

A Always use high reasoning effort for every query to maximize correctness regardless of query type

B **Reserve higher reasoning effort for queries that are genuinely complex or ambiguous, and use lower/default reasoning effort for simple factual queries to minimize unnecessary latency**

C Never use high reasoning effort under any circumstances, since it is never worth the latency cost

D Randomly alternate reasoning effort levels per request to balance cost over time

CORRECT ANSWER **B**

WHY Matching reasoning effort to query complexity is the core accuracy-latency trade-off principle: simple factual queries like office hours gain little accuracy benefit from extended reasoning but pay a real latency cost, so effort should scale with actual task difficulty. Blanket "always high" or "never high" policies both ignore that complexity varies by query, and random alternation has no principled basis.

Q9 **A financial risk-assessment agent currently performs semantic retrieval, then always reranks all retrieved results using a separate reranking model, even when only two documents were retrieved. What adjustment reduces unnecessary latency without harming accuracy?**

A Always rerank regardless of result count, since reranking never affects latency meaningfully

B **Skip or short-circuit the reranking step when the retrieved set is already small enough that reranking provides negligible benefit (e.g., below a defined threshold count)**

C Disable reranking entirely for all queries

D Rerank twice using two different reranking models for every query

CORRECT ANSWER **B**

WHY Reranking adds the most value when there are many candidates to distinguish between; for very small result sets the marginal accuracy benefit is low while the latency cost of an extra model call remains, so conditionally skipping reranking below a threshold is a sound optimization. Reranking does add latency so it's not "negligible" to ignore, disabling it entirely would hurt accuracy on larger result sets, and double reranking only compounds latency for little benefit.

Q10 **A telecom customer-service deployment needs sub-second responses for live chat but also wants high factual accuracy when answering billing disputes, which require retrieving account history. Which combined strategy best balances these goals?**

A Use one single configuration for every message type in the conversation, regardless of whether it's small talk or a billing dispute

B **Classify intent early (e.g., lightweight classifier or fast initial model pass) to route small talk through a fast path and billing disputes through a path with deeper retrieval and validation, even if slightly slower**

C Apply the deepest retrieval and validation pipeline to every message, including greetings, to guarantee consistency

D Apply no retrieval at all to keep every response fast

CORRECT ANSWER **B**

WHY Intent-based routing lets the system allocate latency budget where it matters — deeper retrieval/validation for consequential billing questions while keeping simple exchanges fast — which is the practical way to satisfy both the "sub-second" and "high accuracy" requirements simultaneously. Using one configuration for everything either wastes latency on trivial messages or under-serves accuracy-critical ones, and skipping retrieval entirely would harm accuracy on account-specific disputes.

Q11 **An architect benchmarking a RAG-based legal research tool finds that p50 latency is acceptable (1.2s) but p99 latency spikes to 14s due to occasional retries after validation failures. What should the benchmarking report emphasize?**

A Only the p50 latency, since it represents the "typical" experience

B **Both p50 and tail latency (e.g., p95/p99), since tail latency spikes caused by retries directly affect a meaningful fraction of real users and can indicate systemic reliability issues needing investigation**

C Average latency only, ignoring percentiles entirely

D Whichever single number makes the system look best in the performance report

CORRECT ANSWER B

WHY Performance benchmarking for production systems should report latency distributions, especially tail percentiles (p95/p99), because averages and medians can mask significant portions of users experiencing poor performance due to retries or failures. Reporting only p50 or a cherry-picked favorable number hides real reliability problems that need engineering attention.

Q12 **A manufacturing predictive-maintenance agent must validate that a tool's numeric output (predicted failure probability) falls within a sane range before presenting it to engineers, since occasional model errors produce nonsensical values. The team is debating whether to add this validation given it adds ~100ms per call. What is the best judgment?**

A Skip validation entirely to save 100ms, since engineers can be trusted to notice nonsensical values themselves

B **Add the validation, since 100ms is a small latency cost relative to the risk of presenting clearly incorrect failure-probability data that could lead to a missed or unnecessary maintenance action**

C Add validation but only run it once per day as a batch job disconnected from live requests

D Replace validation with simply rounding all outputs to the nearest whole number

CORRECT ANSWER B

WHY A small, bounded latency cost is generally worth paying when it prevents operationally consequential errors (e.g., a bad failure-probability number driving a costly or unsafe maintenance decision); this is a textbook accuracy-for-latency trade-off that favors accuracy given the stakes. Relying on human review to catch every error is not a reliable control, batch validation disconnected from live use doesn't protect the real-time decision, and rounding does nothing to catch out-of-range or nonsensical values.

Q13 **A retail recommendation agent is evaluated and found to achieve slightly higher accuracy using "full-context retrieval" (loading entire product catalogs into context) compared to targeted semantic search, but at 6x the latency and cost. How should the architect weigh this trade-off for a live shopping assistant?**

A Always choose the higher-accuracy option regardless of cost or latency implications

B **Weigh the marginal accuracy gain against the latency/cost impact in the context of the specific use case; for a live, latency-sensitive shopping assistant, the smaller accuracy gain likely does not justify a 6x cost/latency increase, whereas it might for an offline batch analysis use case**

C Always choose the lower-latency option regardless of the magnitude of the accuracy difference

D Use full-context retrieval but only for the first query in every session

CORRECT ANSWER **B**

WHY Sound accuracy-latency trade-off evaluation is context-dependent: it requires comparing the size of the accuracy gain against the cost in the specific operational setting, and for live, interactive use cases a large latency/cost multiplier is often not justified by a small accuracy improvement, while the same trade-off might be acceptable for offline/batch workloads. Blanket rules to always maximize accuracy or always minimize latency ignore this necessary cost-benefit analysis, and applying full-context retrieval only to the first query lacks any principled justification.

Q14 **An insurance underwriting agent chains three sequential tool calls where each call's input depends on the previous call's output (lookup applicant → fetch risk score → generate premium). A junior engineer proposes parallelizing all three calls to reduce latency. What is the correct architectural judgment?**

A. These calls can always be parallelized safely since parallel execution is always faster

B. These calls have a data dependency chain and cannot be meaningfully parallelized; latency reduction here should instead focus on optimizing each step's individual speed or reducing round-trip overhead between steps

C. Parallelization will work as long as retries are disabled

D. Parallelization is possible by simply guessing the risk score before the lookup completes

CORRECT ANSWER B

WHY Parallel tool execution only provides latency benefit when calls are independent; when each subsequent call needs the prior call's actual output, the dependency chain is inherently sequential, and attempting to parallelize would either fail or produce incorrect results. Disabling retries doesn't resolve a data-dependency problem, and "guessing" an intermediate value defeats the purpose of the lookup and risks generating incorrect premiums.

Q15 **A government agency wants to A/B test whether increasing reasoning effort on its eligibility-determination agent meaningfully improves the correctness of benefit decisions, given the added latency cost. What is the most defensible evaluation methodology?**

A Ask a single reviewer for a subjective opinion on which output "feels" more correct

B **Run both reasoning-effort configurations against a held-out set of cases with known/adjudicated correct outcomes, measure accuracy delta and latency delta, and make the decision based on statistically meaningful results against the agency's accuracy and service-level requirements**

C Deploy both configurations simultaneously to all citizens and resolve any conflicting results by majority vote among users

D Skip evaluation and default to whichever configuration costs less per token

CORRECT ANSWER **B**

WHY A defensible trade-off decision requires an evaluation set with known-correct outcomes, measured accuracy and latency deltas, and a decision framework tied to actual service requirements (this is standard performance benchmarking practice for consequential, regulated decisions). Subjective single-reviewer impressions, uncontrolled simultaneous public deployment with "majority vote," or choosing purely on token cost all lack the rigor needed for a decision affecting real benefit determinations. --